Tampereen ammattikorkeakoulu

Järjestelyalgoritmien suoritys-kykyjen analysointi C++ -kie-lellä

TIRA-lopputyö, Kevät 2026

Tiitus Savonen

SISÄLTÖ

1 JOHDANTO 2
2 AIKATAULUTUS 3
3 DATAN TULKINTA 7
3.1 Yhteenveto 8
3.2 Vertailujen määrä 9
3.3 Muistidatan analyysi 10
3.4 Matemaattinen analyysi 12

1 JOHDANTO

Tämä projekti on toteutettu osana TIRA-kurssin (Tietorakenteen ja algoritmit) lopputyötä. Projektin tavoitteena oli vertailla kolmen erilaisten etsintä- ja lajittelualgoritmien suorituskykyä.

Mittauksia suoritettiin eri kokoisilla datajoukoilla, joissa ilmeni kuinka erilaiset algoritmit toimivat skaalan suurentuessa. Tuloksia verrattiin myös teoreettisiin aikavaativuuksiin, kuten $O(n^2)$ ja $O(NlogN)$.

Tavoittelen työn arvosanaksi 5. Olen lisännyt tarvittavat vaatimukset (Peräkkäisetsintä, binäärietsintä, $O(n^2)$ -lajittelu, $O(NlogN)$ -lajittelu, suorituskykyvertailu) ja tämä raportti sisältää lähdekoodin esittelyä sekä tulosten analysointia.

Linkki GitHub-repoon, josta löytyy myös README -dokumentti:
https://github.com/tksavonen/sorting-algorithm-analyzer

2 AIKATAULUTUS

PÄIVÄ 1 (10.4.) (2h)

Määrittelin ohjelman yleisen rakenteen, käyden läpi vaatimukset ja tavoitteet. Lisäsin pienimuotoisen valikon, käyttäjäsyötteen tunnistuksen ja tynkäfunktioita jokaiselle valinnalle.

```
[1] Linear Search
[2] (Option not implemented)
...
[6] Quit
```

Ohjelman pystyy ajamaan, mutta ihmeempää logiikkaa ei vielä ole lisätty.

PÄIVÄ 2 (12.4) (3h)

Lisäsin ohjelmaan datajoukkogeneraattorin ja tein yleistä käyttöliittymän siistimistä.

```
// final/algorithm.cpp
std::vector<int> generate_data(int n) {
std::vector<int> v(n);

std::random_device rd;
std::mt19937 gen(rd());
std::uniform_int_distribution<> dis(0, 100);

for (int i = 0; i < n; i++)
v[i] = dis(gen);

return v;      }
```

Lineaarihakuun lisätty toimivuus (palauttaa tiedon löytyneestä/löytymättömästä alkiosta.)

```
Enter array size: 1000
Search key: 12
Key found at Position 13!
```

PÄIVÄ 3 (13.4) (1,5h)

Binäärihakuun lisätty toimivuus (varmistaa että datajoukko on lajiteltu, laskee tehdyt operaatiot.)

```
// final/algorithm.cpp
int left = 0, right = v.size() - 1;
int index = -1;
while (left <= right) {
int mid = left + (right - left) / 2;
if (v[mid] == key) {
index = mid;
break; }
if (v[mid] < key)
left = mid + 1;
else
right = mid - 1; }
```

Lisäsin vielä varmistukset virhesyötteitä varten (negatiivinen koko, kaikki paitsi kokonaisluvut.)

```
Your choice > 0
Invalid choice. Try again.
```

PÄIVÄ 4 (14.4) (2h)

O(n²) -algoritmi lisätty (Insertion Sort), eli ensimmäinen lajittelufunktio on valmis.

```
// final/algorithm.cpp
for (int i = 1; i < pop_number; i++) {
int current = v[i];
int j = i - 1;
while (j >= 0 && v[j] > current) {
v[j + 1] = v[j];
j--; } v[j + 1] = current; }
```

Ajanlasku lisätty jokaiseen algoritmioperaatioon.

```
Key not found.
Operation took 0.459 µs.
----------------------------------------
```

PÄIVÄ 5 (15.4) (2h)

O(NlogN) -algoritmi lisätty (Heap Sort). Tämä oli helpoin lisätä, sillä käytin std::kirjastoon sisäänrakennettuja metodeja.

```
// final/algorithm.cpp
std::make_heap(v.begin(), v.end());
std::sort_heap(v.begin(), v.end());
```

Operaatioihin lisätty tarkempaa debug-tietoa, esim. Muistinkäyttö.

```
Memory Usage
Input: 1.953 KB
Auxiliary: 0.000 KB

// final/include/memory.hpp
T* allocate(std::size_t n) {
size_type bytes = n * sizeof(T);
size_type current = allocated.fetch_add(bytes) + bytes;
size_type peak = peak_allocated.load();
```

PÄIVÄ 6 (16.4) (5h)

Algoritmien suorituskykytesti lisätty. Testi pyörittää siis montaa funktiota samalla datajoukolla ja vertailee niiden eroavaisuuksia. Algoritmeina käytetään Insertion, Bubble ja Merge.

Ohjelma laskee operaatioiden määrän ja ajan, jonka algoritmin ajaminen vei.

```
// final/algorithm.cpp
double fastest = std::min({insertion_time, bubble_time, merge_time});
double slowest = std::max({insertion_time, bubble_time, merge_time});
double ratio = slowest / fastest;
// ...
// bubble time implementation
auto v = original;
for (int i = 0; i < pop_number - 1; i++) {
for (int j = 0; j < pop_number - i - 1; j++) {
b_stats.comparisons++;
if (v[j] > v[j + 1]) {
std::swap(v[j], v[j + 1]);
b_stats.assignments += 3; }      }      }
```

export.cpp -kooditiedostolla tallennan benchmark-tiedot exports.csv, josta luon diagrammin käyttäen lyhyttä Python –skriptiä.

```
# final/plot_results.py
df = pd.read_csv("results.csv")
df = df.groupby(["Population", "Algorithm"], as_index=False)["Time(ms)"].mean()

// results.csv
Timestamp,Population,Algorithm,Time(ms),Comparisons,
Assignments,Operations,PeakMemory(KB)
2026-04-17 20:36:38,100,Insertion Sort, 1.896, 396 594,0,0.78125
```

PÄIVÄ 7 (17.4) (3h)

Selvitin bugeja muistilokien ja virheellisten operaatiomäärien kanssa, jotka yleensä johtuivat lajittelemattomista datajoukoista.
Siistin myös käyttöliittymää lisää (värikoodit, tekstin formatointi, pieni latausanimaatio suorituskykytestille...)

```
// final/include/gfx.hpp
// USAGE: std::cout << color::red << "THIS IS RED!";
namespace color {
constexpr const char* red = "\033[31m";
//...
```

Kun olin tyytyväinen tuloksiin, tallensin lähdekoodin julkiseen GitHub-repooni.
Vaikein osa oli varmasti operaatioiden ja muistin seuranta, sekä yhdenmukaiset säännöt jokaiselle algoritmille.

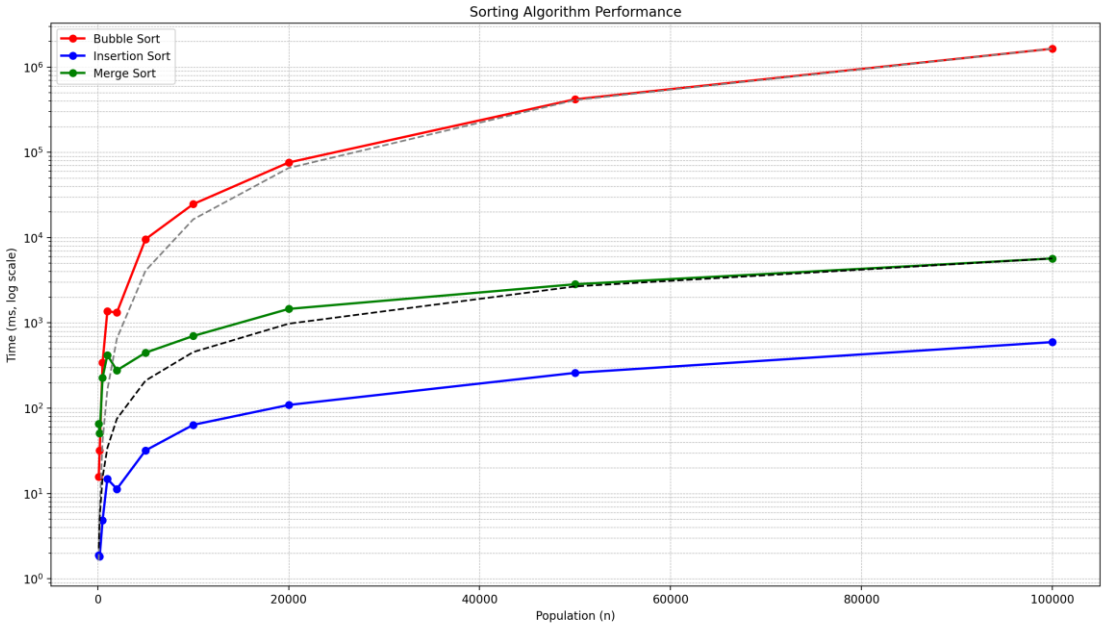
3 DATAN TULKINTA

Tässä osiossa syvennytään Optio 5:n aliohjelmaan—Sorting Performance Comparison—sekä sen suorituskykyyn ja tuloksiin.

3.1 Yhteenveto

Tulokset osoittavat selvästi, että lajittelualgoritmien suorituskyky vaihtelee merkittävästi datajoukon koon kasvaessa. Kolme arvioitua algoritmia—*Bubble Sort*, *Insertion Sort* ja *Merge Sort*—osoittavat kasvuja, jotka ovat johdonmukaisia teoreettisten aikavaatimusten kanssa. Pienillä joukkokooilla (n. ≤ 1000) kaikki algoritmit toimivat hyväksyttävien aikarajojen sisällä. Kuitenkin, kun joukkokoot kasvavat tätä isoimmaksi, ilmenee selkeitä eroja.

- *Bubble Sort* –algoritmista tulee pikaisesti epäpätevä. Ongelmaksi tulee, että algoritmi jää tarkistamaan samoja alkioita uudestaan ja uudestaan.
- *Insertion Sort* –algoritmin suorituskyky pysyy paremmin kohtalaisena.
- *Merge Sort* –algoritmi, joka hyödyntää hajota-ja-hallitse –periaatetta, toimii suurienkin datajoukkojen kanssa.



Tässä diagrammissa näkee kolmen algoritmin kasvukäyrät. Viittauskäyrinä on näkyvissä yksi harmaa *(edustaen O(n²))* ja yksi musta *(edustaen O(NlogN))* katkoviiva. *Merge Sort* seuraa O(NlogN) -käyrää ja *Bubble Sort* että *Insertion Sort* seuraavat läheisesti O(n²)) -käyrää.

Tätä käyrää seuraavien algoritmien suorituskyky eroavat kuitenkin huomattavasti asymptoottisesta suoritusajasta huolimatta.
Alussa näkyvät "piikit" ovat todennäköisesti keskusyksikön digitaalimelun aiheuttamia ja eivät edusta tuloksia yleisellä tasolla.

3.2 Vertailujen määrä

Tulokset osoittavat merkittäviä eroja algoritmien välillä:
Bubble Sort

Populaatio	Aika (ms)	Vertailuoperaatiot
100	15.77	9,900
1000	1360.62	999,000
5000	9584.72	24,995,000

Insertion Sort

Populaatio	Aika (ms)	Vertailuoperaatiot
100	1.896	396
1000	14.813	3,996
5000	31.979	19,996

Merge Sort

Populaatio	Aika (ms)	Vertailuoperaatiot
100	65,375	712
1000	419,18	10,088
5000	447,75	64,008

Tuloksista näkee, että *Bubble Sort* käyttää huomattavasti enemmän vertailuoperaatioita kuin muut algoritmit. Vertailujen määrä kasvaa nopeasti datajoukon koon kasvaessa, mikä vastaa sen teoreettista aikavaativuutta:

$$C(n) = \frac{n(n-1)}{2}$$

Tämä selittää miksi algoritmi muuttuu tehottomaksi suurilla populaatioilla.

Teoreettisesti *Insertion Sort* tekee O(n²)) määrän vertailuja, mutta käytännön tulokset ovat huomattavasti pienempiä.

Algoritmi nimittäin lopettaa prosessin aikaisemmin, jos tarvittava alkio löytyy nopeasti. Tässä tilanteessa data on myös lajiteltu heti generaation jälkeen.

Vertailujen määrä voidaan esittää likimäärin muodossa:

$$C(n) \approx \frac{n^2}{4}$$

Merge Sort eroaa selvästi kahdesta aikaisemmasta algoritmista. Sen vertailujen määrä nousee hitaammin ja se noudattaa logaritmista kasvua.

$$C(n) \approx n \log_2 n$$

Esimerkiksi arvolla *n=5000* saadaan:

$$5000 \cdot \log_2(5000) \approx 61\ 500$$

Tämä vastaa hyvin mitattua arvoa (64 000) mikä vahvistaa teorian paikkansapitävyyden.

3.3 Muistidatan analyysi

Muistinkulutus on tärkeä osa algoritmin tehokkuutta, erityisesti kun kyseessä on suuret tietomäärät ja/tai järjestelmän resurssit ovat rajalliset.

Mittauksissa havaittiin, että muistinkäyttö kasvaa lineaarisesti datajoukon koon mukaan:

Populaatio	Käytetty muisti (KB)
100	0.78
1000	7.81
5000	39.06
10000	78.13
100000	781.25

Tulokset seuraavat siis lineaarista riippuvuutta:

$$RAM\,(n) = k \cdot n$$

jossa *k* on vakio, joka kuvaa yhden alkion vaatimaa muistimäärää.

Mittauksissa havaittu kasvu viittaa siihen, että kokonaismuistinkulutus määräytyy pääasiassa datajoukkokoon mukaan, eikä yksittäisten algoritmien välinen ero näy merkittävästi kokonaislukemissa. Vaikka ohjelma laskee ajoajan kokonaisen käyttömäärän, eikä spesifejä algoritmien sisäisiä käyttömääriä, *Merge Sort* todennäköisesti vaikuttaa eniten muistin kasvuun.
Kasvu on odotettavaa ja ei sisällä yllättäviä eksponentiaalisia piikkejä.

3.4 Matemaattinen analyysi

Tässä alaosiossa tarkastellaan lajittelualgoritmien suorituskykyä matemaattisesta näkökulmasta. Kuten edellä näkyvässä käyrässä, tarkoituksena on verrata tuloksia teoreettisiin malleihin.

Empiirisen datan perusteella, suoritusaika seuraa teoreettisia aikavaatimuksia melko tarkasti:

$$T_{bubble}(n) \approx k_{bubble} \cdot n^2$$

$$T_{insertion}(n) \approx k_{insertion} \cdot n^2$$

$$T_{merge}(n) \approx k_{merge} \cdot n \log^2 n$$

joissa *k* on verrannollisuusvakio, joka edustaa omaa toteutusta ja laitteistoa.

Käyttäen *Bubble Sort* –algoritmia, kun *n*=1000 ja *T*=10^3 ms:

$$K_{bubble} \approx \frac{T}{n^2} = \frac{10^3}{(10^3)\,2} \approx 10^{-3}\ ms\ per\ op.$$

10^-3 =0,001ms

Käyttäen *Merge Sort* –algoritmia, kun *n*=10000 ja *T*=10^3 ms:

$$K_{merge} \approx \frac{T}{n \log^2 n} = \frac{10^3}{10000 \cdot \log_2 10000} \approx 0.0036\ ms\ per\ op.$$

Vaikka *Merge Sort* –algoritmia käyttäen datajoukkoluku on nostettu 10 tuhanteen asti, ero suoritusajassa on vähäpätöinen. Vakioilla on suuri merkitys: *Insertion Sort* on huomattavasti nopeampi kuin *Bubble Sort*, vaikka molemmat ovat neliöllisiä, ja *Merge Sort* on tehokkain suurilla datajoukoilla.

Toinen tapa todentaa *Merge Sort* –algoritmin ylivoimaisuus on löytämällä suurin alaraja.

$$C(n) \geq \log_2(n!)$$

Käyttäen Stirlingin approksimaatiota:

$$\log_2(n!) \approx n \log_2 n - n$$

saamme suurimman alarajan:

$$C(n) \in \Omega(n \log n)$$

Tämä todistaa, että mikään vertailuihin perustuva lajittelualgoritmi ei voi olla nopeampi kuin *(NlogN).*

Toinen merkittävä asia, joka toki vaikuttaa algoritmien suorituskykyyn on käyttäjän laitteisto ja tarkemmin muisti. Vaikka CPU-operaatiot ovat paljon nopeampia, ohjelma tarvitsee välimuistiin varastointiakin toimiakseen.
Voimme käyttää tähän mallia:

$$O\left(\frac{n}{B}\log_{\frac{M}{B}}\frac{n}{B}\right)$$

jossa *n* on datajoukko, *M* on välimuistin koko ja *B* on muistilohkon koko.

Merge Sort käyttää operaatiossaan peräkkäistä pääsykuviota (jatkuvaa kirjoittamista muistisijainteihin, esim. 0x01, 0x02 ...), joka tukee välimuistin pikaista käyttöä. Tähän verrattuna *Bubble Sort*, joka suorittaa jatkuvia vaihtoja ja skannausta, johdattaa huonoon muistiinkäyttöön ja välimuistihuteihin (cache miss).

Kaiken voi tiivistää “kultaiseen” kaavaan:

$$T(n) = \propto \cdot C(n) + \beta \cdot M(n)$$

jossa *C(n)* on vertailujen määrä, *M(n)* on muistioperaatioiden määrä (esim. kirjoitukset, vaihdot) ja α,β ovat laitteistosta riippuvia vakiolukuja.